SIMATS ENGINEERING

ASSESSMENT TOOL III – VISUALIZATION CRITIQUE & REDESIGN

Course: Data Handling and Visualization
Code: DSA0601
CO: CO3
Activity Tool: Visualization Critique & Redesign

QUESTION 1 – Critique and Redesign an ECDF and Q-Q Plot

Dataset

```
delivery <- data.frame(
  Order_ID = paste0("O", sprintf("%02d", 1:20)),
  Warehouse = c(
    rep("Chennai", 5),
    rep("Bengaluru", 5),
    rep("Hyderabad", 5),
    rep("Pune", 5)
  ),
  Delivery_Time_Hrs = c(
    4, 5, 6, 7, 9,
    3, 5, 6, 8, 10,
    6, 7, 8, 12, 15,
    4, 6, 8, 9, 22
  )
)

delivery
```

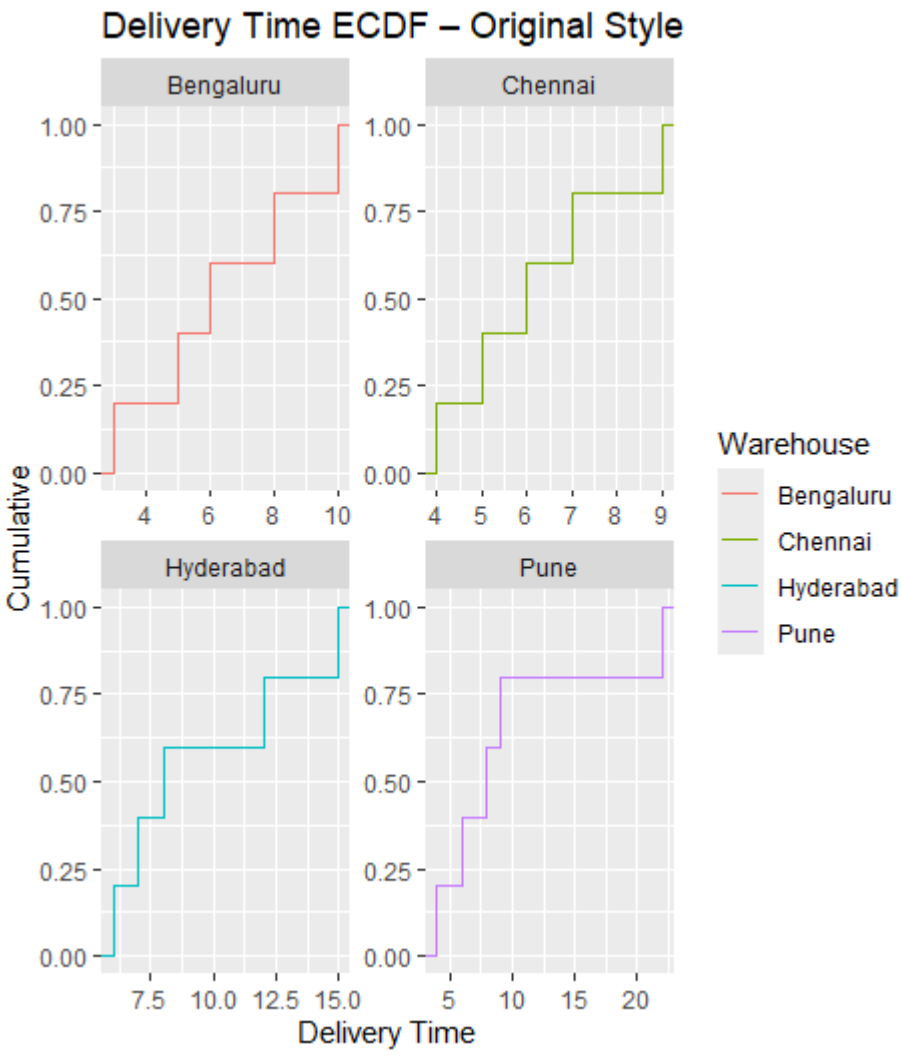
---

1. Original-Style ECDF Plot

```
ggplot(delivery,
```

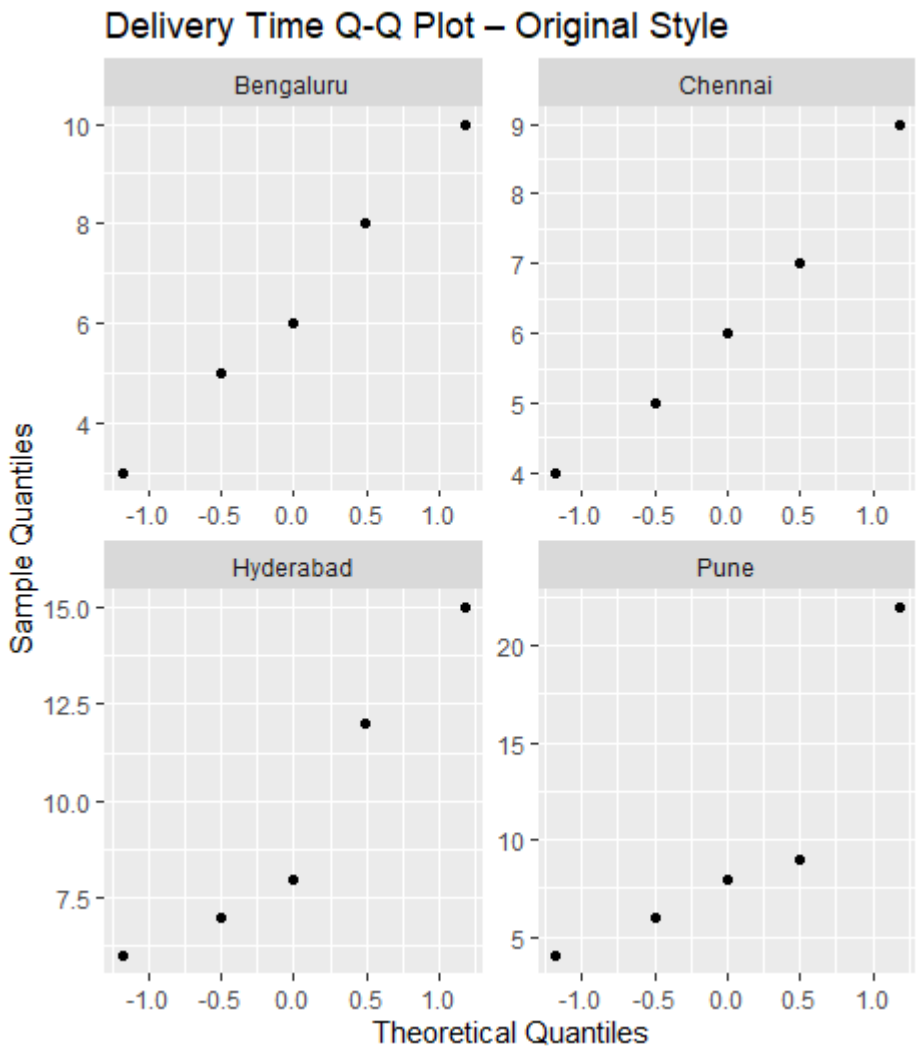
```
    aes(x = Delivery_Time_Hrs, color = Warehouse)) +
 stat_ecdf() +
 facet_wrap(~Warehouse, scales = "free") +
 labs(
  title = "Delivery Time ECDF – Original Style",
  x = "Delivery Time",
  y = "Cumulative"
 ) +
 theme_gray()
```

The use of scales = "free" intentionally represents the inconsistent scaling in the original visualization.



2. Original-Style Q-Q Plot

```
ggplot(delivery, aes(sample = Delivery_Time_Hrs)) +
 stat_qq() +
 facet_wrap(~Warehouse, scales = "free") +
 labs(
  title = "Delivery Time Q-Q Plot – Original Style",
  x = "Theoretical Quantiles",
  y = "Sample Quantiles"
 ) +
 theme_gray()
```



3. Problems in the Original Visualization

Problem 1 – Inconsistent axis scaling

Different warehouses use different axis ranges. This makes visual comparison between warehouses difficult.

Problem 2 – Missing Q-Q reference line

A Q-Q plot should contain a theoretical reference line. Without it, it is difficult to judge how closely the observations follow a normal distribution.

Problem 3 – Excessive gridlines

Heavy gridlines increase visual clutter and distract from the ECDF curves and Q-Q points.

Problem 4 – Poor axis labels

The original labels do not clearly specify units such as hours or cumulative probability.

Problem 5 – Difficult comparison

Using separate scales can make one warehouse appear more or less variable than another even when the difference is partly caused by the scaling.

---

4. Redesigned ECDF Plot

```
ggplot(delivery,
      aes(x = Delivery_Time_Hrs,
          color = Warehouse)) +
 stat_ecdf(linewidth = 1) +
 scale_x_continuous(
   breaks = seq(0, 25, 2),
   limits = c(0, 24)
 ) +
 scale_y_continuous(
   breaks = seq(0, 1, 0.2),
   labels = scales::percent
 ) +
```

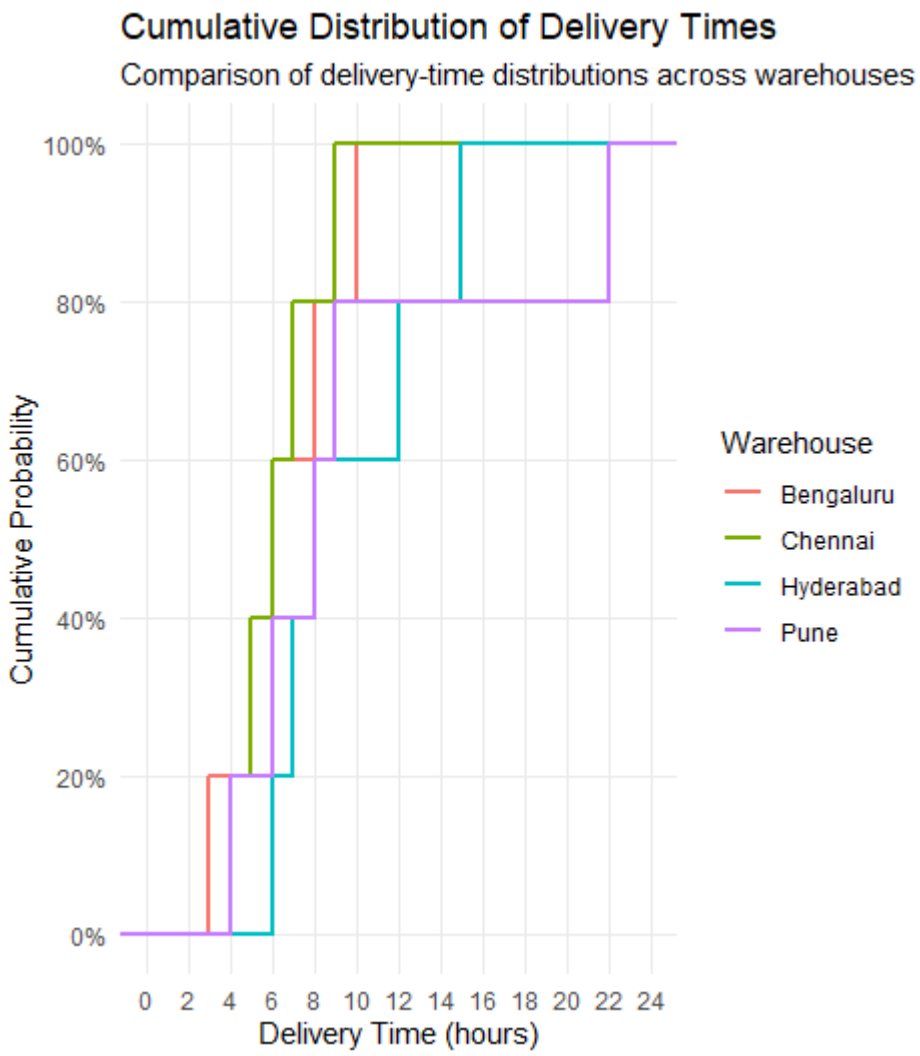
```
 labs(
  title = "Cumulative Distribution of Delivery Times",
  subtitle = "Comparison of delivery-time distributions across warehouses",
  x = "Delivery Time (hours)",
  y = "Cumulative Probability",
  color = "Warehouse"
 ) +
 theme_minimal() +
 theme(
  panel.grid.minor = element_blank()
 )
```

Interpretation

The redesigned ECDF uses:

- Common x-axis scale
- Percentage labels on the y-axis
- Clear axis titles
- Meaningful legend
- Minimal gridlines

The Pune distribution extends to **22 hours**, considerably higher than the other warehouses. This indicates a possible extreme delivery time.

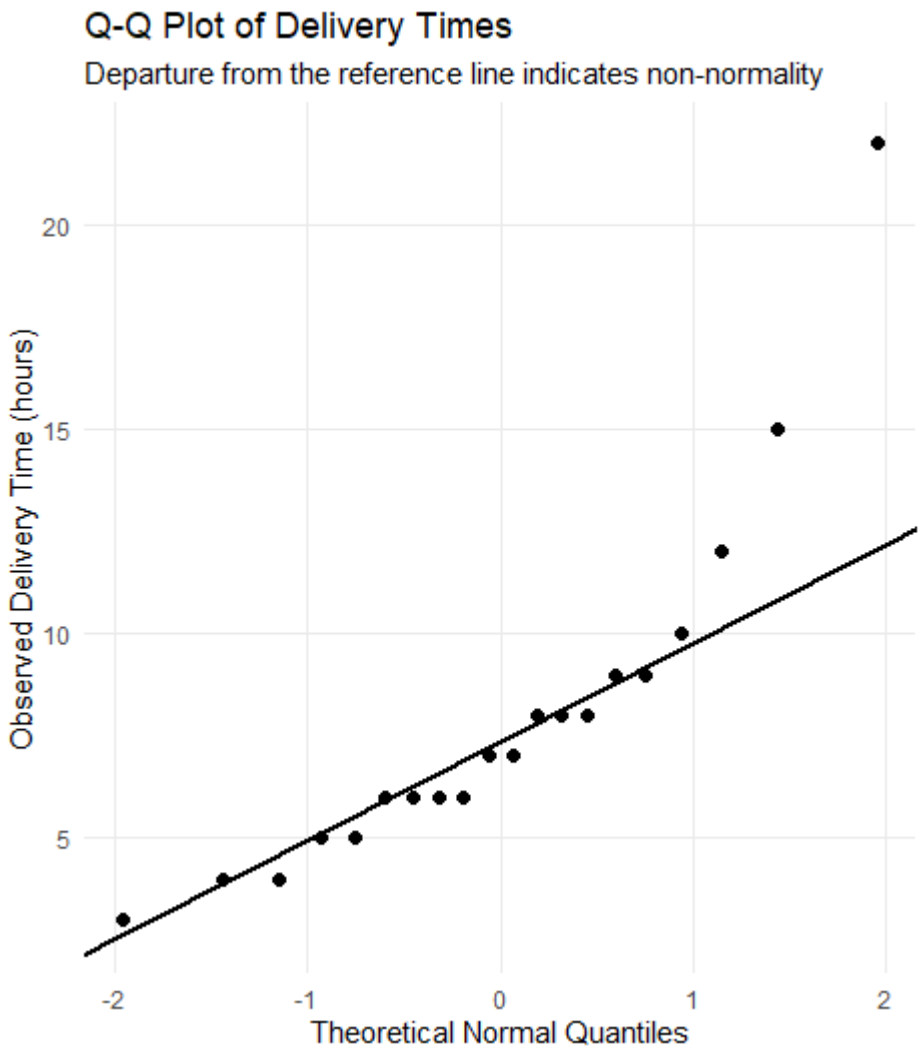


---

5. Redesigned Q-Q Plot

```
ggplot(delivery, aes(sample = Delivery_Time_Hrs)) +
 stat_qq(size = 2) +
 stat_qq_line(linewidth = 1) +
 labs(
  title = "Q-Q Plot of Delivery Times",
  subtitle = "Departure from the reference line indicates non-normality",
  x = "Theoretical Normal Quantiles",
  y = "Observed Delivery Time (hours)"
 ) +
```

```
 theme_minimal() +
 theme(
  panel.grid.minor = element_blank()
 )
```



Q-Q Plot Interpretation

Most observations should approximately follow the reference line if the distribution is normal.

The high value of **22 hours** lies far from the expected pattern and indicates an extreme observation.

The distribution also shows evidence of **right skewness**, because the upper tail extends substantially farther than the lower tail.

Is ECDF Alone Sufficient to Detect Outliers?

No.

An ECDF is useful for understanding:

- Cumulative probability
- Distribution shape
- Percentiles
- Differences between groups

However, it is not the best standalone method for identifying individual outliers.

The ECDF should therefore be paired with:

- Q-Q plot
- Boxplot
- Histogram

For this dataset, the **Q-Q plot and boxplot** are particularly useful for identifying the unusually large value of **22 hours**.

Final conclusion

The redesigned ECDF improves comparison between warehouses, while the Q-Q plot provides a clearer assessment of normality and unusual observations.

QUESTION 2 – Critique and Redesign Many Distributions Using Bean Plots

Dataset

```
marks <- data.frame(
 Student_ID = paste0("T", sprintf("%02d", 1:20)),
 Subject = c(
  rep("Physics", 5),
  rep("Chemistry", 5),
  rep("Mathematics", 5),
  rep("Biology", 5)
 ),
```

```
  Marks = c(
    58, 63, 70, 77, 85,
    52, 60, 68, 74, 82,
    61, 66, 73, 80, 90,
    55, 62, 69, 76, 88
  )
)

marks
```

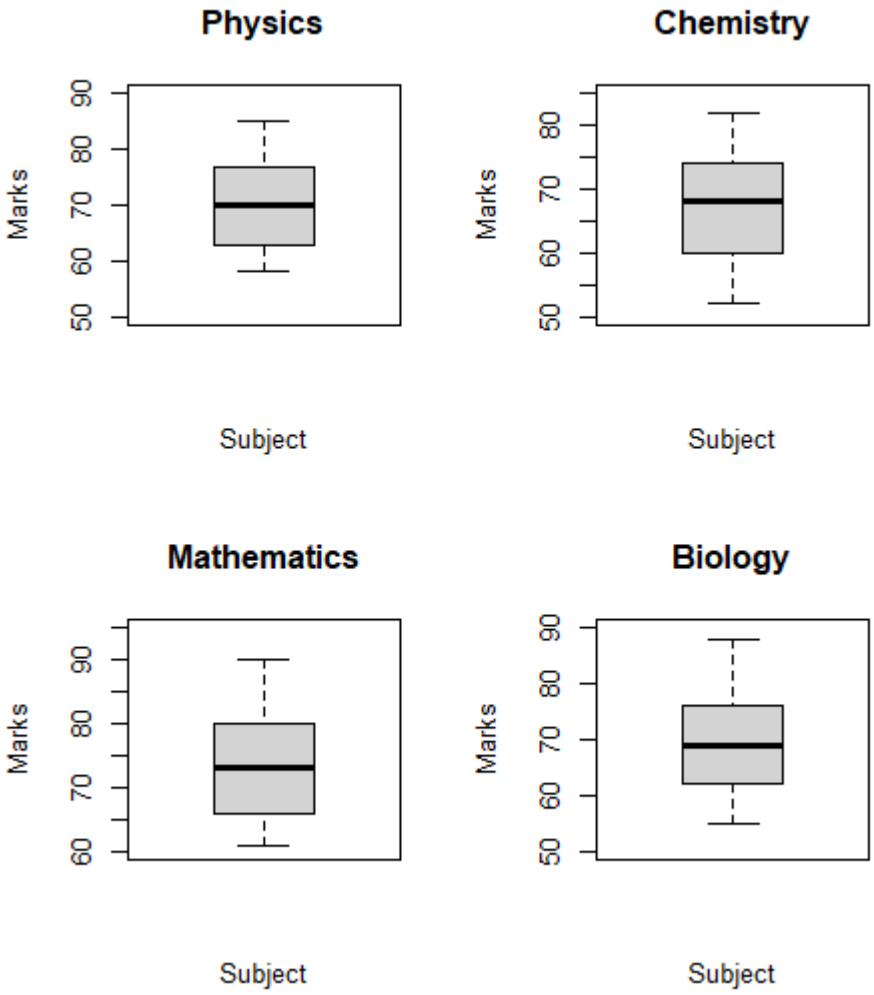
---

1. Original Problematic Boxplots

A representative problematic visualization can be created using separate plots with different y-axis ranges.

```
par(mfrow = c(2, 2))

boxplot(
  Marks ~ Subject,
  data = subset(marks, Subject == "Physics"),
  ylim = c(50, 90),
  main = "Physics",
  ylab = "Marks"
)

boxplot(
  Marks ~ Subject,
  data = subset(marks, Subject == "Chemistry"),
  ylim = c(50, 85),
  main = "Chemistry",
```

```
  ylab = "Marks"
)

boxplot(
  Marks ~ Subject,
  data = subset(marks, Subject == "Mathematics"),
  ylim = c(60, 95),
  main = "Mathematics",
  ylab = "Marks"
)

boxplot(
  Marks ~ Subject,
  data = subset(marks, Subject == "Biology"),
  ylim = c(50, 90),
  main = "Biology",
  ylab = "Marks"
)

par(mfrow = c(1, 1))
```



---

2. Problems with the Original Visualization

Problem 1 – Separate boxplots

Four separate charts make direct comparison difficult.

Problem 2 – Inconsistent y-axis ranges

The subjects use different y-axis limits.

This can visually exaggerate or hide differences between subjects.

Problem 3 – Truncated axes

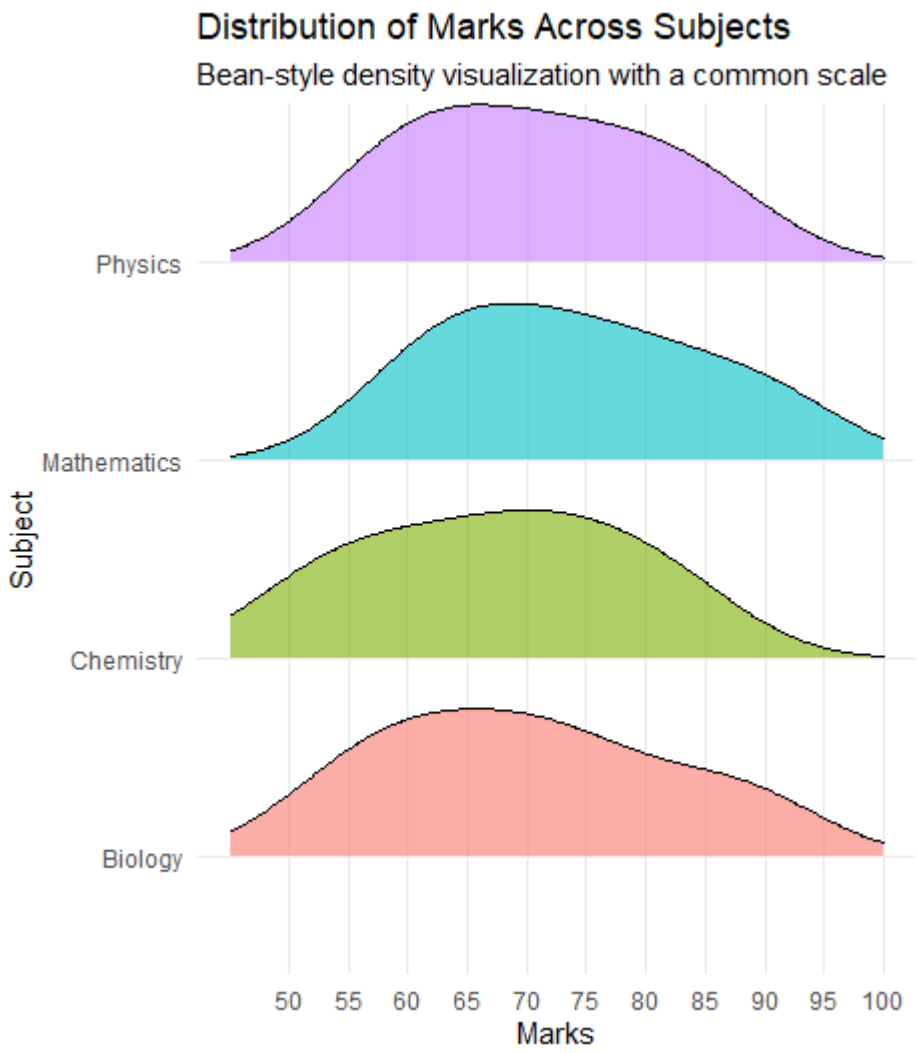
Starting the y-axis at different values can make small differences appear much larger than they actually are.

Problem 4 – Boxplots hide distribution details

A boxplot mainly shows:

- Median
- Quartiles
- Whiskers
- Potential outliers

It does not clearly show the complete distribution of observations.



---

3. Redesigned Bean-Style Visualization

A density-based bean/violin visualization can be created using ridgelines.

```
ggplot(marks,
    aes(
     x = Marks,
```

```
      y = Subject,
      fill = Subject
    )) +
 geom_density_ridges(
  alpha = 0.6,
  scale = 0.8,
  rel_min_height = 0.01
 ) +
 scale_x_continuous(
  limits = c(45, 100),
  breaks = seq(50, 100, 5)
 ) +
 labs(
  title = "Distribution of Marks Across Subjects",
  subtitle = "Bean-style density visualization with a common scale",
  x = "Marks",
  y = "Subject"
 ) +
 theme_minimal() +
 theme(
  legend.position = "none",
  panel.grid.minor = element_blank()
 )
```

Alternative Violin Plot

If ggridges is unavailable, use a violin plot:

```
ggplot(marks,
     aes(x = Subject, y = Marks, fill = Subject)) +
```

```
 geom_violin(
  trim = FALSE,
  alpha = 0.6
 ) +
 geom_boxplot(
  width = 0.12,
  fill = "white"
 ) +
 geom_jitter(
  width = 0.08,
  size = 2
 ) +
 scale_y_continuous(
  limits = c(45, 100),
  breaks = seq(50, 100, 5)
 ) +
 labs(
  title = "Marks Distribution by Subject",
  x = "Subject",
  y = "Marks"
 ) +
 theme_minimal() +
 theme(
  legend.position = "none",
  panel.grid.minor = element_blank()
 )
```

---

4. Common Scale

The redesigned visualization uses a common scale from approximately **45 to 100 marks**.

This allows all four subjects to be compared fairly.

---

5. Interpretation

Highest concentration of marks

Physics shows the greatest concentration because it has the smallest overall range:

- Physics: 58–85 → **27 marks**
- Chemistry: 52–82 → **30 marks**
- Mathematics: 61–90 → **29 marks**
- Biology: 55–88 → **33 marks**

Physics therefore has the narrowest spread.

Greatest spread

Biology has the greatest spread:

88 - 55 = 33 marks

Therefore:

Highest concentration: Physics
Greatest spread: Biology

Mathematics has the highest median at approximately **73 marks**, indicating the strongest central performance.

QUESTION 3 – Critique and Redesign Proportion Visualizations

Dataset

```
downloads <- data.frame(
  City = c(
    "Chennai", "Chennai", "Chennai",
    "Mumbai", "Mumbai", "Mumbai",
    "Delhi", "Delhi", "Delhi",
    "Kolkata", "Kolkata", "Kolkata"
```

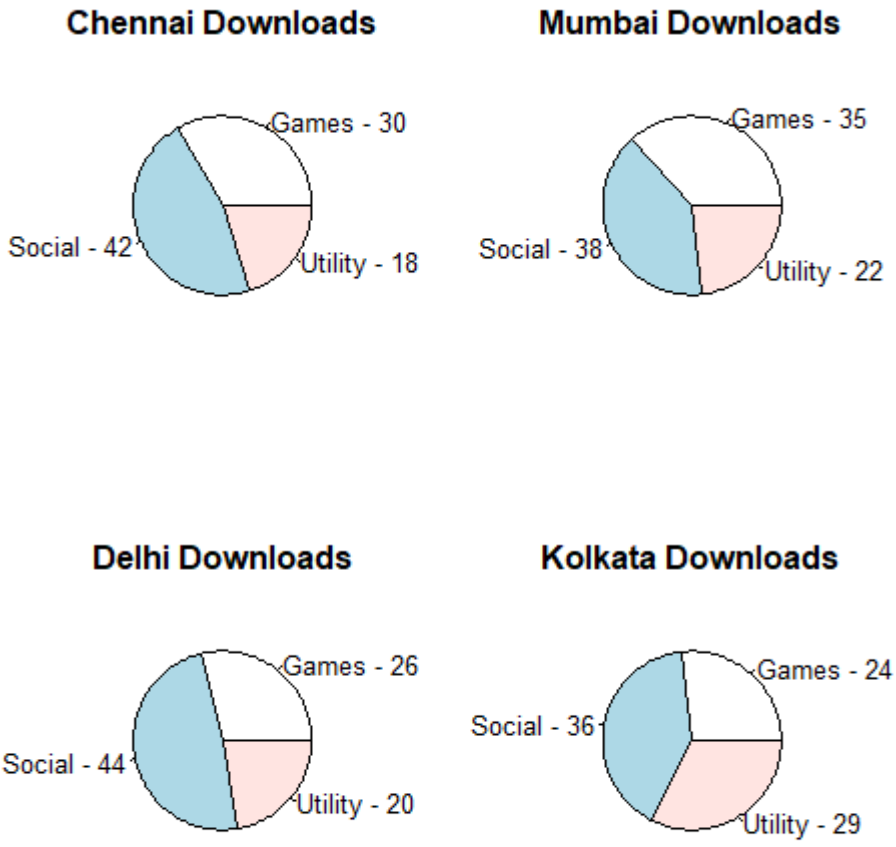
```
  ),
  App_Category = c(
    "Games", "Social", "Utility",
    "Games", "Social", "Utility",
    "Games", "Social", "Utility",
    "Games", "Social", "Utility"
  ),
  Downloads = c(
    30, 42, 18,
    35, 38, 22,
    26, 44, 20,
    24, 36, 29
  )
)

downloads
```

---

1. Original Misleading Pie Charts

```
par(mfrow = c(2, 2))

cities <- unique(downloads$City)

for (city in cities) {

  temp <- subset(downloads, City == city)

  pie(
    temp$Downloads,
```

```
  labels = paste(
    temp$App_Category,
    temp$Downloads,
    sep = " - "
  ),
  main = paste(city, "Downloads"),
  explode = c(0.1, 0, 0.15)
 )
}

par(mfrow = c(1, 1))
```



---

2. Problems with the Original Visualization

Problem 1 – Exploded slices

Exploding slices can visually emphasize certain categories even when their numerical values are not substantially different.

Problem 2 – Mismatched colors

Using different colors in different pie charts makes it difficult to track the same category across cities.

For example, Games should have the same visual identity in every city.

Problem 3 – Difficult angle comparison

Humans are less accurate at comparing angles than lengths.

Problem 4 – Inconsistent labeling

Different labels or positions increase cognitive effort.

Problem 5 – Multiple pie charts

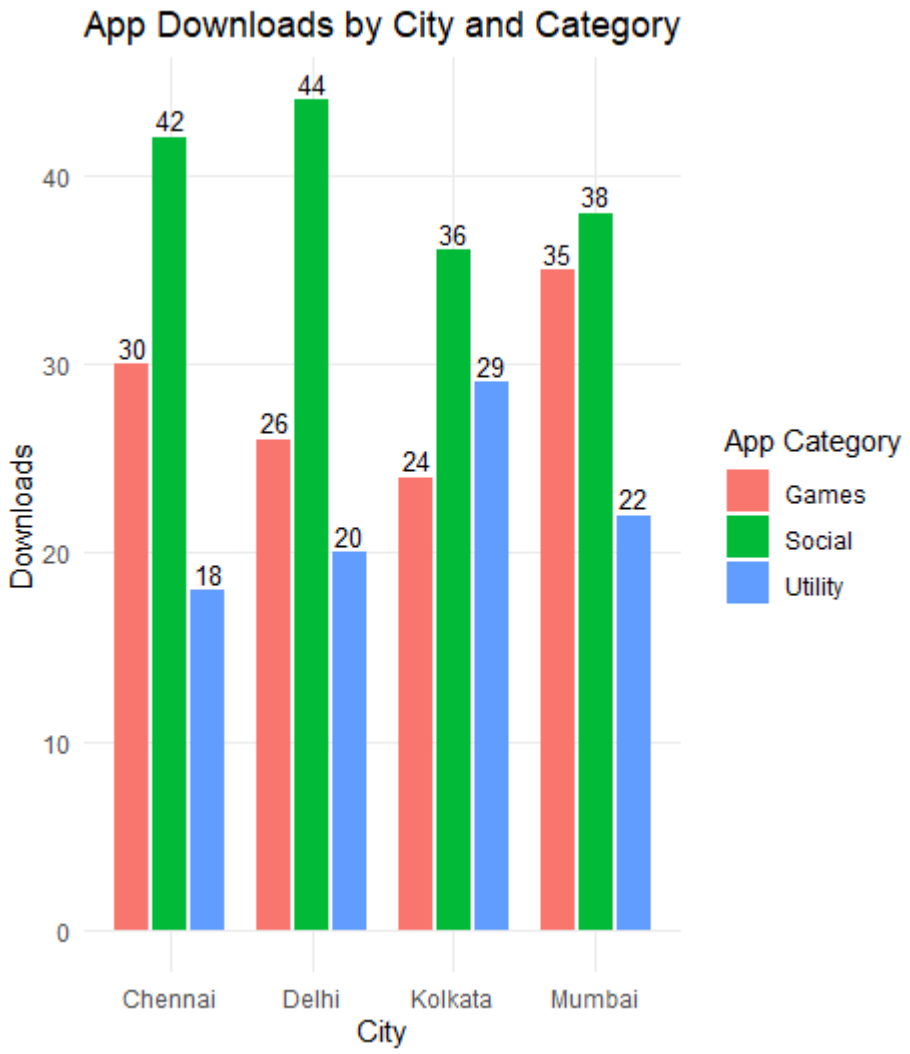
Comparing four pie charts is difficult because each category occupies a different angle and location.

---

3. Redesign – Side-by-Side Bar Chart

```
ggplot(downloads,
    aes(
      x = City,
      y = Downloads,
      fill = App_Category
    )) +
 geom_col(
   position = position_dodge(width = 0.8),
   width = 0.7
 ) +
 geom_text(
```

```
  aes(label = Downloads),
  position = position_dodge(width = 0.8),
  vjust = -0.3,
  size = 3.5
) +
labs(
  title = "App Downloads by City and Category",
  x = "City",
  y = "Downloads",
  fill = "App Category"
) +
theme_minimal() +
theme(
  panel.grid.minor = element_blank()
)
```



---

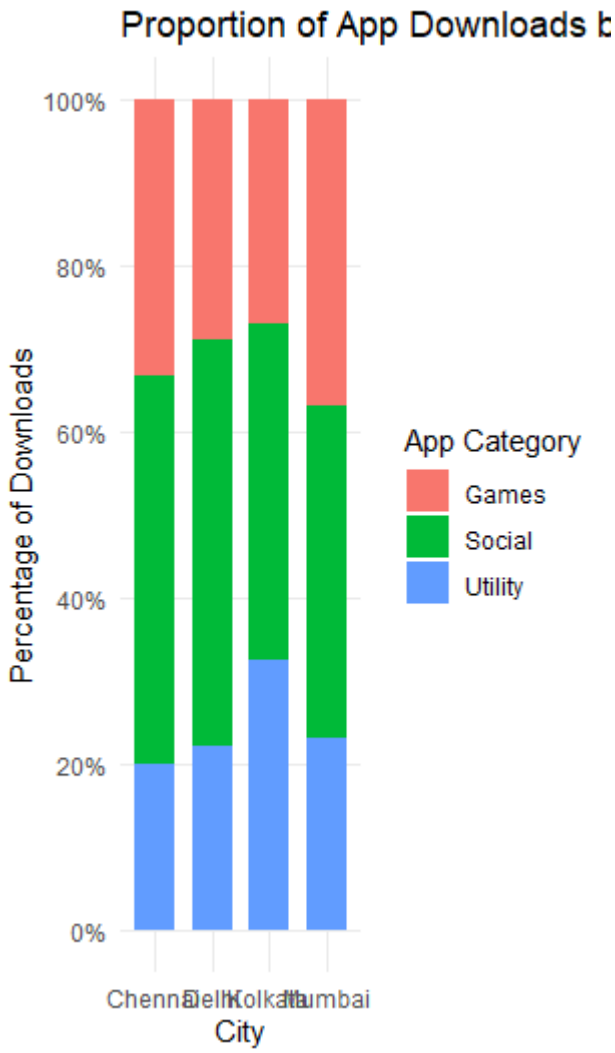
4. Redesign – 100% Stacked Bar Chart

First calculate the proportions:

```
downloads_prop <- downloads %>%
 group_by(City) %>%
 mutate(
  Proportion = Downloads / sum(Downloads)
 ) %>%
 ungroup()


downloads_prop
```

Now create the chart:

```
ggplot(
  downloads_prop,
  aes(
    x = City,
    y = Proportion,
    fill = App_Category
  )
) +
  geom_col(width = 0.7) +
  scale_y_continuous(
    labels = scales::percent,
    breaks = seq(0, 1, 0.2)
  ) +
  labs(
    title = "Proportion of App Downloads by City",
    x = "City",
    y = "Percentage of Downloads",
    fill = "App Category"
  ) +
  theme_minimal() +
  theme(
    panel.grid.minor = element_blank()
  )
```



---

5. Comparison of the Two Redesigns

Side-by-side bar chart

Advantages:

- Easy comparison of individual categories
- Exact values are easier to compare
- Common baseline improves accuracy

Disadvantage:

- Takes more horizontal space
- Overall city composition is less immediately visible

100% stacked bar chart

Advantages:

- Every city has a total of 100%
- Clearly communicates proportions
- Makes differences in city composition easy to see
- More compact than multiple pie charts

Disadvantage:

- Categories other than the first segment are harder to compare precisely because they do not share a common baseline.

Recommendation

For **city-wise proportions**, the **100% stacked bar chart** is the better overall visualization because every city is normalized to 100%, making the composition directly comparable.

For comparing the **exact number of downloads in each category**, the side-by-side bar chart is better.

QUESTION 4 – Critique and Redesign Nested Proportions

Dataset

```
expenditure <- data.frame(
  Campus = c(
    "Campus A", "Campus A", "Campus A", "Campus A",
    "Campus B", "Campus B", "Campus B", "Campus B",
    "Campus C", "Campus C", "Campus C", "Campus C"
  ),
  Department = c(
    "Engineering", "Engineering",
    "Management", "Management",
    "Engineering", "Engineering",
    "Management", "Management",
    "Engineering", "Engineering",
```

```
  "Management", "Management"
 ),
 Expense_Type = c(
  "Faculty Salary", "Lab Equipment",
  "Faculty Salary", "Guest Lectures",
  "Faculty Salary", "Lab Equipment",
  "Faculty Salary", "Guest Lectures",
  "Faculty Salary", "Lab Equipment",
  "Faculty Salary", "Guest Lectures"
 ),
 Amount = c(
  62000, 34000, 28000, 15000,
  54000, 40000, 24000, 17000,
  66000, 31000, 26000, 19000
 )
)

expenditure
```

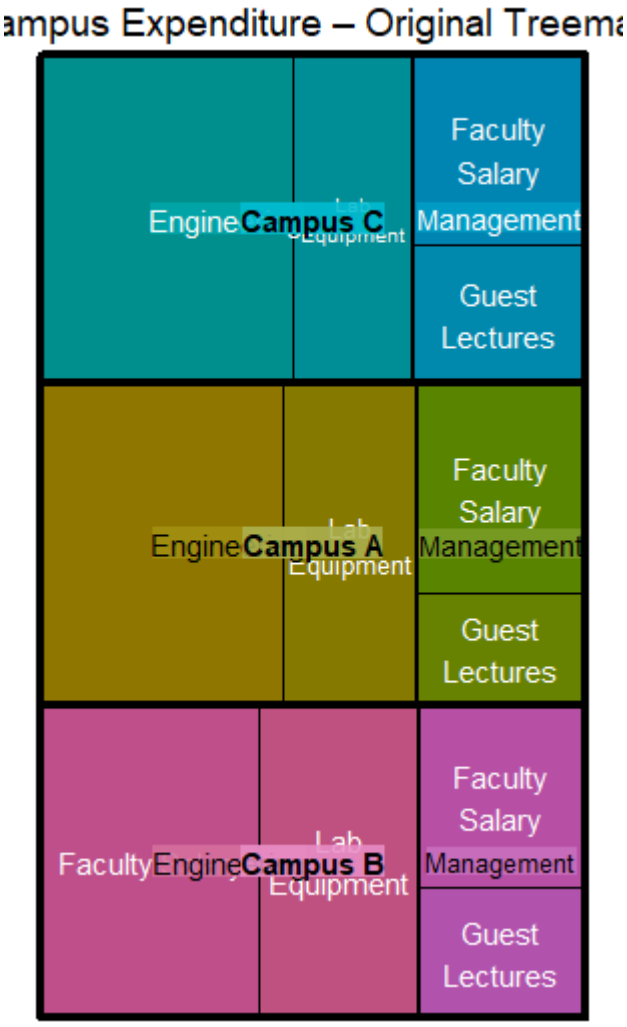
---

1. Original Treemap

```
treemap(
 expenditure,
 index = c(
  "Campus",
  "Department",
  "Expense_Type"
 ),
 vSize = "Amount",
```

title = "Campus Expenditure – Original Treemap"

)



---

2. Problems with the Original Treemap

Problem 1 – Small areas have unreadable labels

Small expense categories have very little space for text.

Problem 2 – Weak hierarchy

The relationship between:

Campus

↓

Department

↓

Expense Type

is not always visually obvious.

Problem 3 – Poor color encoding

If colors do not distinguish hierarchy levels or departments consistently, the viewer may misunderstand the structure.

Problem 4 – Area comparison is difficult

Comparing similarly sized rectangles is less accurate than comparing bars.

---

3. Improved Treemap

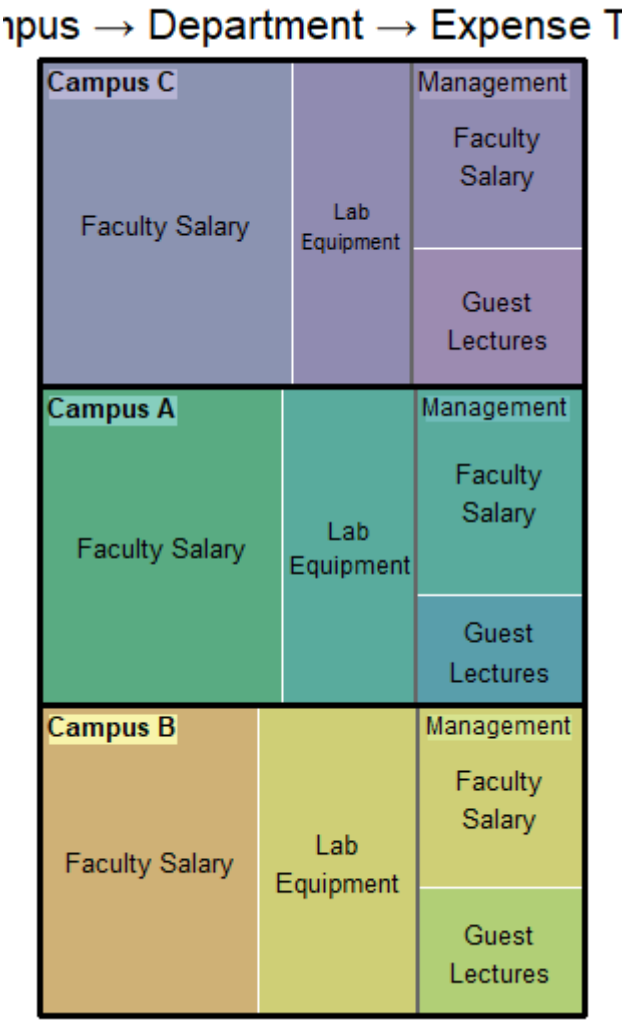
```
treemap(
 expenditure,
 index = c(
  "Campus",
  "Department",
  "Expense_Type"
 ),
 vSize = "Amount",
 type = "index",
 palette = "Set3",
 title = "Campus → Department → Expense Type",
 fontsize.title = 16,
 fontsize.labels = 10,
 border.col = c("black", "grey40", "white"),
 border.lwds = c(3, 2, 1),
 align.labels = list(
  c("left", "top"),
  c("left", "top"),
```

```
    c("center", "center")
  )
)
```

Design improvements

The redesigned treemap uses:

- Clear hierarchy
- Common area scale
- Stronger borders between hierarchy levels
- Consistent labeling
- Improved title
- Meaningful color differentiation



---

4. Sunburst Alternative

Prepare hierarchical nodes:

```
campus_nodes <- expenditure %>%
 group_by(Campus) %>%
 summarise(Amount = sum(Amount), .groups = "drop") %>%
 transmute(
  id = Campus,
  label = Campus,
  parent = "",
  value = Amount
 )

department_nodes <- expenditure %>%
 group_by(Campus, Department) %>%
 summarise(Amount = sum(Amount), .groups = "drop") %>%
 mutate(
  id = paste(Campus, Department, sep = " - "),
  label = Department,
  parent = Campus,
  value = Amount
 ) %>%
 select(id, label, parent, value)

expense_nodes <- expenditure %>%
 mutate(
  id = paste(Campus, Department, Expense_Type, sep = " - "),
  parent = paste(Campus, Department, sep = " - "),
```

```
  label = Expense_Type,
  value = Amount
 ) %>%
 select(id, label, parent, value)

sunburst_data <- bind_rows(
 campus_nodes,
 department_nodes,
 expense_nodes
)
Create the sunburst:
plot_ly(
 sunburst_data,
 ids = ~id,
 labels = ~label,
 parents = ~parent,
 values = ~value,
 type = "sunburst",
 branchvalues = "total"
) %>%
 layout(
  title = "Campus Expenditure Sunburst"
 )
```

Campus Expenditure Sunburst

---

5. Treemap vs Sunburst

Treemap

Advantages:

- Efficient use of screen space
- Easy to see large spending categories
- Good for comparing overall amounts
- Suitable for executive dashboards

Disadvantages:

- Small categories may be difficult to read
- Hierarchy is represented using nested rectangles

Sunburst

Advantages:

- Hierarchy is visually explicit
- Campus → Department → Expense Type is represented through rings
- Good for exploring nested structures

Disadvantages:

- Comparing areas and angles is more difficult
- Small segments can become crowded
- Exact amount comparison is harder

Board Presentation Recommendation

For a **board presentation**, I recommend the **redesigned treemap**.

The treemap uses space efficiently and allows executives to quickly identify the largest expenditure areas.

For this dataset:

- Campus A total = **139,000**
- Campus B total = **135,000**
- Campus C total = **142,000**

Therefore, **Campus C has the highest total expenditure**.

The treemap is preferable for quickly identifying major spending categories, while the sunburst is more appropriate when the main objective is exploring the hierarchy.

QUESTION 5 – Critique and Redesign Flow-Based Proportions

Dataset

```
career <- data.frame(
  Course = c(
    "Data Science", "Data Science", "Data Science",
    "Web Development", "Web Development", "Web Development",
    "Cloud Computing", "Cloud Computing", "Cloud Computing",
```

```
  "UI/UX Design", "UI/UX Design", "UI/UX Design"
 ),
 Certification = c(
  "Yes", "Yes", "No",
  "Yes", "Yes", "No",
  "Yes", "Yes", "No",
  "Yes", "Yes", "No"
 ),
 Job_Role = c(
  "Analyst", "Engineer", "Analyst",
  "Frontend Dev", "Backend Dev", "Frontend Dev",
  "DevOps Engineer", "Cloud Analyst", "Support Engineer",
  "Product Designer", "UX Researcher", "Product Designer"
 ),
 Students = c(
  22, 14, 9,
  19, 11, 7,
  17, 10, 6,
  13, 8, 5
 )
)

career
```

---

1. Create Sankey Diagram

We first calculate the Course → Certification links.

```
course_cert <- career %>%
 group_by(Course, Certification) %>%
```

```
  summarise(
    Students = sum(Students),
    .groups = "drop"
  )
```

Calculate Certification → Job Role links:

```
cert_job <- career %>%
  group_by(Certification, Job_Role) %>%
  summarise(
    Students = sum(Students),
    .groups = "drop"
  )
```

Create nodes:

```
course_nodes <- data.frame(
  name = unique(career$Course)
)

cert_nodes <- data.frame(
  name = unique(career$Certification)
)

job_nodes <- data.frame(
  name = unique(career$Job_Role)
)

nodes <- bind_rows(
  course_nodes,
  cert_nodes,
  job_nodes
```

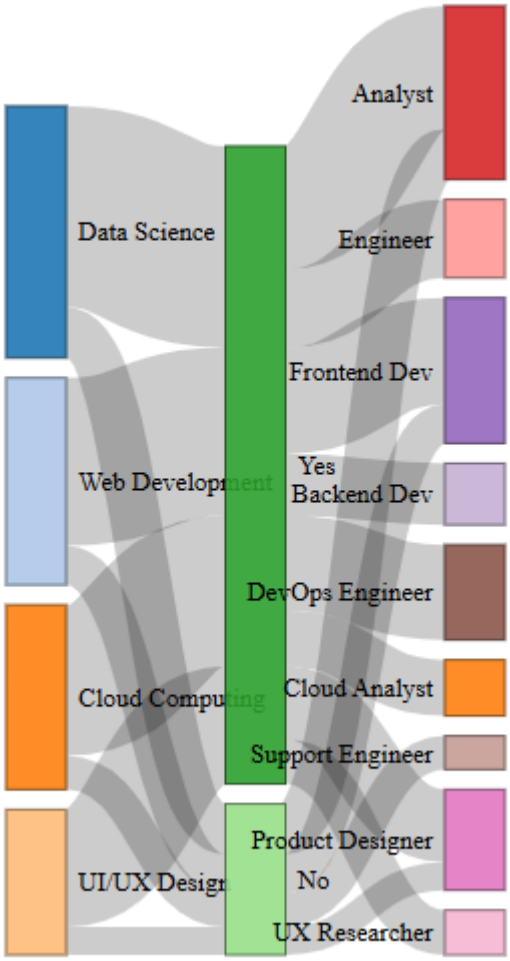
```
)

nodes
Create Course → Certification links:
links1 <- course_cert %>%
 transmute(
  source = match(Course, nodes$name) - 1,
  target = match(Certification, nodes$name) - 1,
  value = Students
 )
Create Certification → Job Role links:
links2 <- cert_job %>%
 transmute(
  source = match(Certification, nodes$name) - 1,
  target = match(Job_Role, nodes$name) - 1,
  value = Students
 )
Combine links:
links <- bind_rows(links1, links2)

links
Create the Sankey diagram:
sankeyNetwork(
 Links = links,
 Nodes = nodes,
 Source = "source",
 Target = "target",
 Value = "value",
```

NodeID = "name",

fontSize = 13,

nodeWidth = 30,

sinksRight = TRUE



)

---

2. Problems in the Original Sankey Diagram

Problem 1 – Near-identical colors

Similar colors make different flows difficult to distinguish.

Problem 2 – Unlabelled flows

Without labels, the viewer cannot immediately determine how many students follow each path.

Problem 3 – Difficult node comparison

Poor node widths make it difficult to compare student totals.

Problem 4 – Weak visual hierarchy

The three stages should clearly communicate:

Course → Certification → Job Role

Problem 5 – Flow crossing

Multiple flows can cross or converge, making individual paths difficult to follow.

Problem 6 – Certification labels are ambiguous

Using only "Yes" and "No" without a clear stage heading can cause confusion.

---

3. Redesigned Sankey Diagram

A better approach is to label the certification nodes more clearly.

```
career2 <- career %>%
 mutate(
  Certification_Label = ifelse(
   Certification == "Yes",
   "Certified",
   "Not Certified"
  )
 )
```

Create the redesigned links:

```
course_cert2 <- career2 %>%
 group_by(Course, Certification_Label) %>%
 summarise(
  Students = sum(Students),
  .groups = "drop"
 )
```

```
cert_job2 <- career2 %>%
 group_by(Certification_Label, Job_Role) %>%
 summarise(
  Students = sum(Students),
  .groups = "drop"
 )
```

Create nodes:

```
nodes2 <- data.frame(
 name = unique(c(
  career2$Course,
  career2$Certification_Label,
  career2$Job_Role
 ))
)
```

Create links:

```
links_course_cert <- course_cert2 %>%
 transmute(
  source = match(Course, nodes2$name) - 1,
  target = match(Certification_Label, nodes2$name) - 1,
  value = Students
 )

links_cert_job <- cert_job2 %>%
 transmute(
  source = match(Certification_Label, nodes2$name) - 1,
  target = match(Job_Role, nodes2$name) - 1,
  value = Students
 )
```

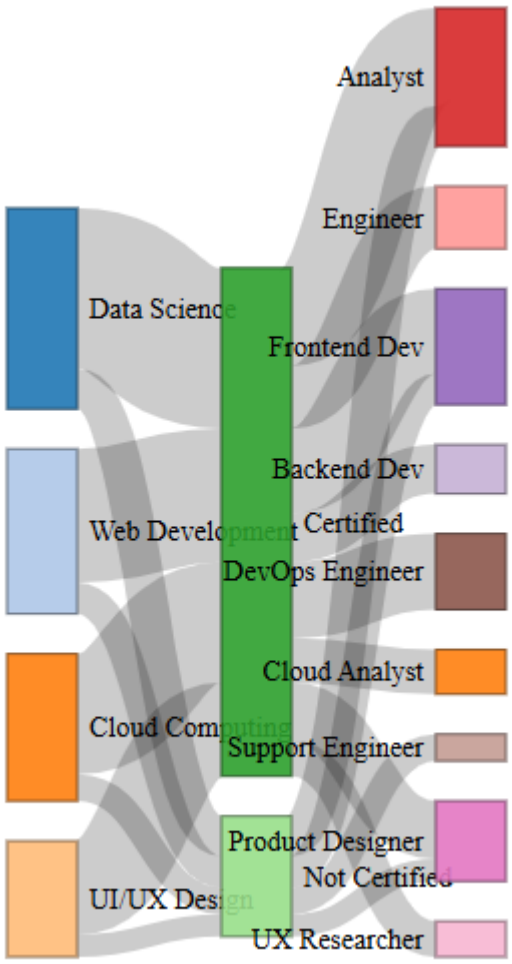
```
links2 <- bind_rows(
  links_course_cert,
  links_cert_job
)
```

Redesigned Sankey:

```
sankeyNetwork(
  Links = links2,
  Nodes = nodes2,
  Source = "source",
  Target = "target",
  Value = "value",
  NodeID = "name",
  fontSize = 14,
  nodeWidth = 35,
  nodePadding = 20,
  sinksRight = TRUE,
  iterations = 64
)
```

The redesigned version has:

- Clear certification labels
- Meaningful node names
- Consistent flow widths
- Larger node widths
- Better spacing
- Clear Course → Certification → Job Role hierarchy



---

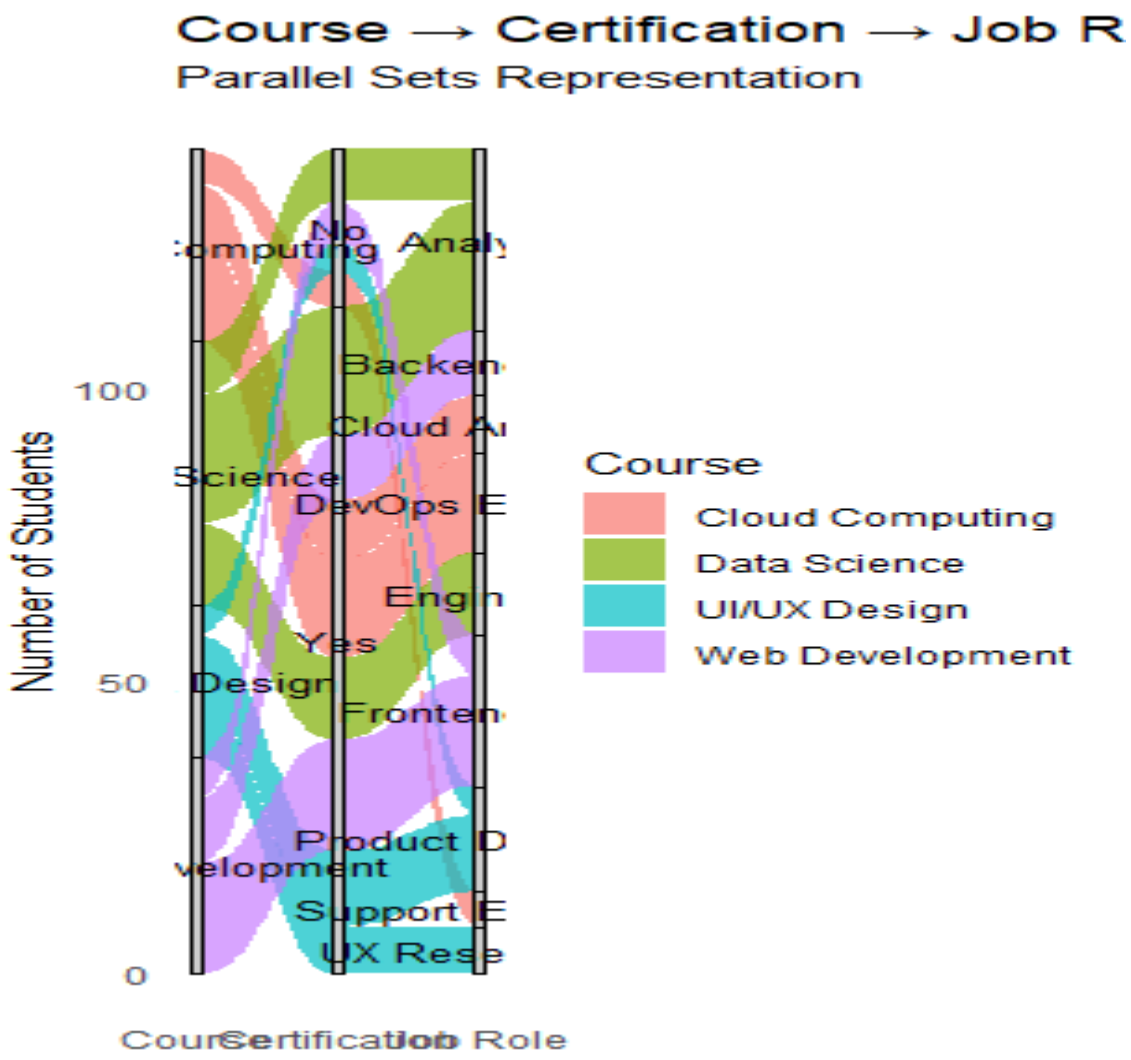
4. Parallel Sets Alternative

The ggalluvial package can be used to create a parallel-sets style visualization.

```
ggplot(
  career,
  aes(
    axis1 = Course,
    axis2 = Certification,
    axis3 = Job_Role,
    y = Students
  )
```

```
) +
 geom_alluvium(
  aes(fill = Course),
  alpha = 0.7,
  width = 1/12
 ) +
 geom_stratum(
  width = 1/12,
  fill = "grey80",
  color = "black"
 ) +
 geom_text(
  stat = "stratum",
  aes(label = after_stat(stratum)),
  size = 3.5
 ) +
 scale_x_discrete(
  limits = c(
   "Course",
   "Certification",
   "Job Role"
  ),
  expand = c(0.05, 0.05)
 ) +
 labs(
  title = "Course → Certification → Job Role",
  subtitle = "Parallel Sets Representation",
  x = NULL,
```

```
  y = "Number of Students",
  fill = "Course"
) +
theme_minimal() +
theme(
  panel.grid = element_blank()
)
```



---

5. Sankey vs Parallel Sets

Sankey Diagram

Advantages:

- Excellent for showing flow quantities
- Flow width directly represents student count
- Easy to identify major career pathways
- Effective for Course → Certification → Job Role relationships

Disadvantages:

- Flow crossings can cause clutter
- Small flows can become difficult to trace

Parallel Sets

Advantages:

- Clearly separates categorical dimensions
- Good for comparing multiple categorical variables
- Shows the relationship between Course, Certification and Job Role
- Can be easier to read when many categories exist

Disadvantages:

- Individual flow paths may be less visually intuitive
- Flow quantities are still represented through band widths

Major Career Flows

The largest individual flow in the dataset is:

Data Science → Certified → Analyst = 22 students

Other major flows include:

- Web Development → Certified → Frontend Dev = **19**
- Cloud Computing → Certified → DevOps Engineer = **17**
- Data Science → Certified → Engineer = **14**
- UI/UX Design → Certified → Product Designer = **13**

Therefore, the major career pathways are concentrated around certified students.

Final Recommendation

For this dataset, the **Sankey diagram is the better visualization**.

The main reason is that the task is specifically about communicating **flows** from:

Course

↓

Certification

↓

Job Role

Sankey diagrams represent the number of students using the width of each flow, allowing the viewer to immediately identify major pathways.

The parallel sets plot is useful as an alternative when the focus is on comparing categorical dimensions rather than emphasizing the magnitude of individual flows.

OVERALL CONCLUSION

The visualization redesign process improves the original charts by applying appropriate visual encoding, common scales, clear labels, meaningful hierarchy and reduced visual clutter.

Question 1

The **ECDF + Q-Q plot** combination is better than ECDF alone because the ECDF shows cumulative distribution while the Q-Q plot helps identify departures from normality and unusual observations.

The **22-hour Pune delivery time** is an important extreme observation.

Question 2

The **bean/violin-style visualization** provides a better representation of distribution shape than four separate boxplots.

- Highest concentration: **Physics**
- Greatest spread: **Biology**
- Highest median: **Mathematics**

Question 3

The **100% stacked bar chart** is preferred for comparing city-wise proportions, while the side-by-side bar chart is better for comparing exact category values.

Exploded pie slices and inconsistent colors can exaggerate visual differences and make category tracking difficult.

Question 4

The **redesigned treemap** is recommended for the board presentation because it communicates hierarchical expenditure efficiently and makes large spending areas easy to identify.

- Campus A = **139,000**
- Campus B = **135,000**
- Campus C = **142,000**

Campus C has the highest expenditure.

Question 5

The **Sankey diagram** is recommended because it directly communicates the magnitude of Course → Certification → Job Role flows.

The largest individual flow is:

Data Science → Certified → Analyst = 22 students.
